

PARQUEO · LOGICIEL ITSM AUTO-HÉBERGÉ

Documentation d'installation

Prérequis, déploiement, configuration, sauvegarde, mise à jour et dépannage.

VERSION 0.1.0 30 JUILLET 2026 GITHUB.COM/LCANET2/PARQUEO

Installation, configuration et exploitation. Public visé : la personne qui déploie et maintient l'application.

- | | |
|--|--|
| 1. Prérequis | 6. Configuration de l'inventaire automatique |
| 2. Installation en production | 7. Sauvegarde et restauration |
| 3. Référence des variables d'environnement | 8. Mise à jour |
| 4. Configuration du SSO Microsoft Entra ID | 9. Exploitation |
| 5. Configuration du collecteur email | 10. Dépannage |

1. Prérequis

Composant	Version	Remarque
Node.js	20 LTS minimum, 22 recommandé	
PostgreSQL	14 minimum	
Reverse proxy	nginx, Caddy, Traefik...	requis (voir §2.5)

Optionnels, activés seulement s'ils sont configurés : un serveur **SMTP** (notifications), une boîte **IMAP** (collecteur email), un **tenant Microsoft Entra ID** (SSO).

Ressources : l'API est un processus Node mono-thread léger. Pour quelques dizaines d'utilisateurs simultanés, 1 vCPU et 1 Go de RAM suffisent, PostgreSQL compris. L'espace disque est dicté par les pièces jointes (10 Mo par fichier au maximum).

2. Installation en production

Exemple pour une machine Debian/Ubuntu, application déployée dans `/opt/parqueo`, servie par nginx sur `https://parqueo.exemple.fr`.

2.1 Utilisateur système et code

```
sudo adduser --system --group --home /opt/parqueo parqueo
sudo -u parqueo git clone https://github.com/Lcanet2/parqueo.git /opt/parqueo
```

2.2 Base de données

```
sudo -u postgres psql -c "CREATE USER parqueo WITH PASSWORD '<mot-de-passe-solide>';"
sudo -u postgres psql -c "CREATE DATABASE parqueo OWNER parqueo;"
```

2.3 API

```
cd /opt/parqueo/server
sudo -u parqueo npm ci
sudo -u parqueo npx prisma generate
sudo -u parqueo cp .env.example .env
sudo -u parqueo nano .env # voir §3
sudo chmod 600 .env
sudo -u parqueo npx prisma migrate deploy
sudo -u parqueo npm run seed
```

Installez **toutes** les dépendances (`npm ci`, sans `--omit=dev`) : la CLI Prisma qui applique les migrations est une dépendance de développement. `migrate deploy` applique les migrations existantes sans jamais réinitialiser la base — c'est la commande de production. Le seed crée un compte `admin@parqueo.local` / `admin1234`, à changer au premier démarrage.

Générer un `JWT_SECRET` solide :

```
node -e "console.log(require('crypto').randomBytes(48).toString('base64url'))"
```

2.4 Client

```
cd /opt/parqueo/client
sudo -u parqueo npm ci
sudo -u parqueo npm run build # produit client/dist
```

`dist/` est un ensemble de fichiers statiques : aucun processus Node ne tourne pour le front.

2.5 Reverse proxy

Le client appelle `/api` en **relatif** : le proxy doit servir le front et l'API **sur la même origine**. C'est la seule contrainte de déploiement structurante.

```
server {
    listen 443 ssl http2;
    server_name parqueo.exemple.fr;

    ssl_certificate      /etc/letsencrypt/live/parqueo.exemple.fr/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/parqueo.exemple.fr/privkey.pem;

    # Pièces jointes : 10 Mo côté application ; la limite nginx par défaut (1 Mo)
    # les rejeterait avant d'atteindre Node.
    client_max_body_size 12M;

    root /opt/parqueo/client/dist;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;    # SPA : repli sur index.html
    }

    location /api {
        proxy_pass http://127.0.0.1:4000;
        proxy_http_version 1.1;
        proxy_set_header Host                $host;
        proxy_set_header X-Real-IP           $remote_addr;
        proxy_set_header X-Forwarded-For     $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto  $scheme;
    }
}

server {
    listen 80;
    server_name parqueo.exemple.fr;
    return 301 https://$host$request_uri;
}
```

Le jeton SSO transite en query string (`?sso_token=...`) : si vous journalisez les URL complètes, filtrez ce paramètre.

2.6 Service systemd

`/etc/systemd/system/parqueo.service` :

```
[Unit]
Description=Parqueo API
After=network.target postgresql.service
Requires=postgresql.service

[Service]
Type=simple
User=parqueo
Group=parqueo
# Impératif : uploads/ est résolu relativement au répertoire de travail.
WorkingDirectory=/opt/parqueo/server
ExecStart=/usr/bin/node src/index.js
Restart=on-failure
RestartSec=5
NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=full
ReadWritePaths=/opt/parqueo/server/uploads

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now parqueo
sudo systemctl status parqueo
```

Le `.env` est lu par `dotenv` depuis le répertoire de travail (pas besoin de `EnvironmentFile`).

WorkingDirectory est critique : les pièces jointes sont écrites dans `uploads/` en chemin relatif ; démarrer ailleurs rend les fichiers existants introuvables.

2.7 Premier démarrage

1. ouvrir `https://parqueo.exemple.fr` ;
2. se connecter avec `admin@parqueo.local` / `admin1234` ;
3. **changer le mot de passe**, ou créer un admin nominatif puis supprimer le compte de seed ;
4. créer les équipes et les catégories ;
5. parcourir Paramètres.

3. Référence des variables d'environnement

Fichier `server/.env`. Seules les quatre premières sont nécessaires au démarrage.

3.1 Cœur

Variable	Défaut	Rôle
<code>DATABASE_URL</code>	—	chaîne de connexion PostgreSQL (requis)
<code>JWT_SECRET</code>	—	clé de signature des jetons (requis) — la changer déconnecte tout le monde
<code>PORT</code>	<code>4000</code>	port d'écoute de l'API
<code>CLIENT_ORIGIN</code>	—	origine autorisée par CORS ; en production, l'URL publique du site
<code>APP_URL</code>	<code>http://localhost:\$PORT</code>	URL publique de l'API, utilisée dans les liens des emails

3.2 Email sortant

Variable	Défaut	Rôle
<code>SMTP_HOST</code>	vide	si vide, aucun email n'est envoyé
<code>SMTP_PORT</code>	<code>587</code>	
<code>SMTP_USER</code> / <code>SMTP_PASS</code>	vide	authentification, omise si <code>SMTP_USER</code> est vide
<code>SMTP_FROM</code>	—	expéditeur, ex. <code>Parqueo <no-reply@exemple.fr></code>

3.3 Clôture automatique

Variable	Défaut	Rôle
<code>AUTO_CLOSE_DAYS</code>	<code>7</code>	valeur initiale seulement ; une fois enregistré dans Paramètres, c'est la base qui fait foi

3.4 SSO Microsoft Entra ID

Variable	Défaut	Rôle
<code>SSO_TENANT_ID</code>	vide	identifiant d'annuaire — les trois premières activent le SSO
<code>SSO_CLIENT_ID</code>	vide	identifiant d'application
<code>SSO_CLIENT_SECRET</code>	vide	secret client
<code>SSO_REDIRECT_URI</code>	<code>APP_URL</code> + <code>/api/auth/sso/callback</code>	doit correspondre exactement à l'app registration

Variable	Défaut	Rôle
<code>SSO_ALLOWED_DOMAINS</code>	vide	garde-fou : domaines email acceptés, séparés par des virgules
<code>SSO_POST_LOGIN_URL</code>	<code>CLIENT_ORIGIN</code>	où renvoyer le navigateur après connexion

3.5 Collecteur email

Variable	Défaut	Rôle
<code>IMAP_HOST</code>	vide	si vide, le collecteur est désactivé
<code>IMAP_PORT</code>	<code>993</code>	
<code>IMAP_USER</code> / <code>IMAP_PASS</code>	—	identifiants de la boîte
<code>IMAP_TLS</code>	<code>true</code>	<code>false</code> pour désactiver TLS
<code>IMAP_POLL_SECONDS</code>	<code>60</code>	intervalle de relève, plancher à 15 s
<code>IMAP_CATEGORY_ID</code>	première catégorie	catégorie des tickets créés par email

3.6 Inventaire automatique

Variable	Défaut	Rôle
<code>INVENTORY_TOKEN</code>	vide	si vide, les endpoints <code>/api/inventory</code> sont désactivés ; sinon, secret présenté par les agents/scripts
<code>INTUNE_ENABLED</code>	<code>false</code>	connecteur Intune (permission Graph requise, voir §6)
<code>INTUNE_SYNC_HOURS</code>	<code>6</code>	cadence de la synchro Intune (0 = seulement à la demande)
<code>SNMP_ENABLED</code>	<code>false</code>	scan réseau SNMP
<code>SNMP_RANGES</code>	vide	plages CIDR à scanner, séparées par des virgules
<code>SNMP_COMMUNITY</code>	<code>public</code>	communauté SNMP v2c (lecture seule)
<code>SNMP_SCAN_HOURS</code>	<code>0</code>	cadence du scan (0 = seulement à la demande)

4. Configuration du SSO Microsoft Entra ID

4.1 Côté Entra (portail Azure)

1. Inscriptions d'applications → Nouvelle inscription.
2. Nom : `Parqueo`. Types de comptes : **comptes de cet annuaire uniquement**.
3. URI de redirection, type **Web** : `https://parqueo.exemple.fr/api/auth/sso/callback`.
4. Relever l'**ID d'application (client)** et l'**ID d'annuaire (locataire)**.

5. **Certificats et secrets → Nouveau secret client.** Copier la **valeur** (elle n'est plus affichée ensuite) et noter la date d'expiration.
6. **Autorisations d'API :** `openid`, `profile`, `email` (délégées Microsoft Graph, généralement déjà présentes).

4.2 Côté Parqueo

```
SSO_TENANT_ID="<id-annuaire>"
SSO_CLIENT_ID="<id-application>"
SSO_CLIENT_SECRET="<valeur-du-secret>"
SSO_REDIRECT_URI="https://parqueo.exemple.fr/api/auth/sso/callback"
SSO_ALLOWED_DOMAINS="exemple.fr"
```

Redémarrer l'API : le bouton « Se connecter avec Microsoft » apparaît automatiquement sur la page de connexion.

4.3 Comportement

- Un compte inconnu est **créé au premier login** en rôle `user` ; les élévations de rôle restent manuelles.
- Un compte SSO n'a pas de mot de passe local : le formulaire classique le refuse.
- Les comptes locaux fonctionnent en parallèle — accès de secours si le SSO tombe.
- **Le secret client expire.** Notez la date : à l'expiration, le SSO échoue d'un bloc. Gardez au moins un compte admin local.

5. Configuration du collecteur email

Dédiez une boîte au support (ex. `support@exemple.fr`) — le collecteur **marque les messages comme lus** et relève toute la boîte de réception.

```
IMAP_HOST="imap.exemple.fr"
IMAP_PORT=993
IMAP_USER="support@exemple.fr"
IMAP_PASS="<mot-de-passe>"
IMAP_POLL_SECONDS=60
IMAP_CATEGORY_ID=1
SMTP_FROM="Support <support@exemple.fr>"
```

Fonctionnement :

- un email d'un **utilisateur connu** crée un ticket ; un expéditeur inconnu est ignoré (et journalisé) ;

- une réponse dont le sujet contient `Ticket #n` devient un **commentaire** — ce qui rend les notifications répondables directement ;
- les adresses `SMTP_FROM` et `IMAP_USER` sont ignorées (anti-boucle).

Conseil : utilisez la **même adresse** pour `SMTP_FROM` et `IMAP_USER`, pour que les réponses aux notifications reviennent dans le collecteur. Au démarrage, les journaux affichent `[collecteur] boîte support@exemple.fr relevée toutes les 60s`.

6. Configuration de l'inventaire automatique

L'inventaire peut se remplir seul via trois moyens, combinables. Tous convergent vers le même traitement (déduplication par `uuid` puis numéro de série ; les champs saisis à la main ne sont jamais écrasés). Guide d'usage côté interface : documentation fonctionnelle §9.

6.1 Push par agent ou script (token partagé)

```
INVENTORY_TOKEN="<long-secret-aléatoire>"
```

Sans lui, `POST /api/inventory` et `/api/inventory/glpi` répondent **404**. Les appelants le présentent en en-tête `X-Parqueo-Token` (ou `Authorization: Bearer`).

- **Script maison** → `POST /api/inventory`, corps JSON `{ uuid?, serial?, name?, type?, manufacturer?, model?, os?, cpu?, ramMb?, diskGb?, software? }` (au moins `uuid` ou `serial`). Idéal en tâche planifiée (cron, GPO). Le tableau `software` remplace la liste connue ; absent, elle est conservée.
- **Agent GLPI** → installez l'agent (MSI/GPO, paquet Linux, pkg macOS) et pointez sa cible sur l'endpoint GLPI de Parqueo, avec le token en en-tête :

```
# agent.cfg
server = https://parqueo.exemple.fr/api/inventory/glpi
httpd-headers = X-Parqueo-Token: <long-secret-aléatoire>
```

Parqueo accepte le format d'inventaire natif (JSON, compressé zlib/gzip ou non). Testez sur un poste pilote avant de généraliser.

6.2 Connecteur Microsoft Intune (sans agent)

Pour un parc enrôlé dans Intune, Parqueo interroge Microsoft Graph — rien à déployer sur les postes.

1. Sur l'app registration Entra du SSO (§4), ajoutez la permission **applicative** Graph `DeviceManagementManagedDevices.Read.All`, puis **accordez le consentement administrateur**.
2. Dans le `.env` (les `SSO_*` doivent déjà être renseignés) :

```
INTUNE_ENABLED=true
INTUNE_SYNC_HOURS=6
```

3. Redémarrez l'API : une synchro part au démarrage, puis toutes les `INTUNE_SYNC_HOURS`. Un admin peut aussi la lancer depuis *Inventaire* → *Synchroniser Intune*.

6.3 Scan réseau SNMP (sans agent)

Pour les équipements sans agent (imprimantes, switches, NAS, onduleurs) :

```
SNMP_ENABLED=true
SNMP_RANGES="192.168.1.0/24,10.0.0.0/28"
SNMP_COMMUNITY="public"
SNMP_SCAN_HOURS=0
```

Le serveur doit joindre les équipements en **UDP 161** (pare-feux). Le scan se lance depuis *Inventaire* → *Scanner le réseau*, et en tâche de fond si `SNMP_SCAN_HOURS` > 0. Préférez un adressage **fixe** : l'identité de déduplication d'un équipement réseau dérive de son nom système.

7. Sauvegarde et restauration

Deux choses à sauvegarder : la base et `server/uploads/`. Le reste se réinstalle depuis Git.

7.1 Sauvegarde

```
#!/bin/sh
# /opt/parqueo/backup.sh
set -e
DEST=/var/backups/parqueo
STAMP=$(date +%F)
mkdir -p "$DEST"

sudo -u postgres pg_dump -Fc parqueo > "$DEST/parqueo-$STAMP.dump"
tar czf "$DEST/uploads-$STAMP.tar.gz" -C /opt/parqueo/server uploads

find "$DEST" -type f -mtime +30 -delete
```

Dans la crontab root : `15 2 * * * /opt/parqueo/backup.sh`.

N'oubliez pas `server/.env` (il contient `JWT_SECRET` et les secrets SMTP/SSO) : sauvegardez-le séparément, dans un coffre — **pas** dans la même archive que la base.

7.2 Restauration

```
sudo systemctl stop parqueo
sudo -u postgres dropdb parqueo
sudo -u postgres createdb parqueo -O parqueo
sudo -u postgres pg_restore -d parqueo /var/backups/parqueo/parqueo-2026-07-23.dump
sudo tar xzf /var/backups/parqueo/uploads-2026-07-23.tar.gz -C /opt/parqueo/server
sudo chown -R parqueo:parqueo /opt/parqueo/server/uploads
sudo systemctl start parqueo
```

Restaurer la base **sans** les uploads laisse des pièces jointes référencées mais absentes du disque : les deux vont ensemble.

8. Mise à jour

```
sudo systemctl stop parqueo

cd /opt/parqueo
sudo -u parqueo git pull

cd server
sudo -u parqueo npm ci
sudo -u parqueo npx prisma generate
sudo -u parqueo npx prisma migrate deploy

cd ../client
sudo -u parqueo npm ci
sudo -u parqueo npm run build

sudo systemctl start parqueo
```

Sauvegardez **avant** toute mise à jour embarquant une migration : Prisma ne propose pas de retour arrière automatique.

9. Exploitation

9.1 Journaux

Tout part sur la sortie standard, donc dans journald :

```
sudo journalctl -u parqueo -f
```

Préfixes utiles :

Préfixe	Origine
[collecteur]	collecteur IMAP
[clôture auto]	clôture automatique horaire
[intune] / [snmp]	synchro Intune / scan réseau
[mail]	échec d'envoi SMTP
[workflow « ... »]	action de workflow en échec

9.2 Supervision

- Sonde de vie : `GET /api/health` → `{"ok":true}`.
- Surveillez l'espace disque de `server/uploads/` : **rien n'est jamais purgé**, y compris les fichiers des tickets supprimés.
- La table `ticket_comments` porte messages **et** journal d'événements : elle croît vite.

9.3 Tâches planifiées internes

Aucune crontab à créer : la clôture automatique (horaire), le collecteur email (selon

`IMAP_POLL_SECONDS`), et — si activées — la synchro Intune (`INTUNE_SYNC_HOURS`) et le scan SNMP (`SNMP_SCAN_HOURS`) tournent dans le processus de l'API et s'arrêtent avec lui.

10. Dépannage

Symptôme	Cause probable	Correction
<code>EADDRINUSE :::4000</code>	une autre instance tourne déjà	arrêter le processus ou changer <code>PORT</code>
Page blanche, 404 sur les routes internes	nginx sans repli SPA	ajouter <code>try_files \$uri \$uri/ /index.html</code>
Toutes les requêtes API en 401	<code>JWT_SECRET</code> modifié, ou jeton expiré (7 j)	se reconnecter
403 « Accès refusé » sur une page admin	rôle insuffisant, ou rôle changé sans reconnexion	se déconnecter/reconnecter : le rôle est figé dans le jeton
413 à l'envoi d'une pièce jointe	<code>client_max_body_size</code> nginx trop bas	passer à 12M

Symptôme	Cause probable	Correction
« Fichier manquant ou type non autorisé »	extension hors liste blanche (images, PDF, docs bureautiques, archives, logs)	convertir ou compresser le fichier
Aucun email reçu	<code>SMTP_HOST</code> vide	les journaux affichent <code>[mail non envoyé...]</code>
Emails entrants ignorés	expéditeur inconnu de Parqueo	créer le compte, ou l'importer en masse
Boucle de tickets créés par email	<code>SMTP_FROM</code> $\neq$ <code>IMAP_USER</code> et boîte auto-répondeuse	aligner les deux adresses
Bouton SSO absent	une des trois variables <code>SSO_*</code> manque	vérifier <code>GET /api/auth/config</code>
<code>AADSTS50011</code> (redirect URI)	l'URI ne correspond pas à l'app registration	aligner <code>SSO_REDIRECT_URI</code> au caractère près
Pièces jointes introuvables après un déplacement	processus lancé hors de <code>server/</code>	corriger <code>WorkingDirectory</code>
Migration bloquée	schéma divergent	<code>npx prisma migrate status</code> ; ne jamais lancer <code>migrate dev</code> en production